

A3: Small Object Detection Using YOLO

CSCI 611 – Spring 2026

Arushi Tewari

1. Dataset Description and Preprocessing

1.1 Dataset

The dataset used for this assignment is the Traffic and Road Signs Computer Vision Dataset, sourced from Roboflow Universe (<https://universe.roboflow.com/usmanchaudhry622-gmail-com/traffic-and-road-signs>). It contains 10,000 annotated images across 29 traffic sign classes, including speed limits, directional signs, warning signs, and regulatory signs such as No Entry, Give Way, Stop Sign, and Pedestrian Crossing.

The dataset was downloaded directly using the Roboflow Python SDK in YOLOv8 format, which provides pre-structured train, validation, and test splits along with a data.yaml configuration file ready for use with the Ultralytics YOLO framework.

1.2 Dataset Structure

- Training set: ~8,000 images
- Validation set: ~1,000 images
- Test set: 1,024 images (approximately 10% of total)
- Annotation format: YOLO format (.txt files with normalized bounding box coordinates)
- Classes: 29 traffic sign categories

1.3 Preprocessing Steps

The dataset was pre-annotated by the original authors, so no manual annotation was required. The following preprocessing steps were applied during training:

- Image resizing to 640x640 pixels for training
- Data augmentation including random scaling, flipping, mosaic augmentation, and brightness/contrast adjustments (enabled via augment=True in YOLO training config)
- Normalization of bounding box coordinates to YOLO format (x_center, y_center, width, height — all relative to image dimensions)

2. Challenges in Detecting Small Objects

Small object detection presents several unique challenges compared to standard object detection:

- **Low pixel density:** Small and distant traffic signs occupy very few pixels in an image, providing limited visual information for the model to learn from.
- **Class similarity:** Many traffic signs share similar shapes (circular, triangular) and colors, making it difficult to distinguish between classes at low resolution.
- **Background clutter:** Real-world images contain complex backgrounds including trees, buildings, and other vehicles that can cause false detections.
- **Scale variation:** Traffic signs appear at vastly different sizes depending on the distance from the camera, requiring the model to be robust across multiple scales.
- **Lighting and occlusion:** Signs may be partially occluded or captured under poor lighting conditions, reducing detection confidence.
- **Pre-trained model limitations:** General-purpose YOLO models trained on COCO classes have no knowledge of specific traffic sign categories, resulting in near-zero baseline performance on this task.

3. Experimental Results

3.1 Baseline: Pre-trained YOLOv8n (Before Fine-tuning)

The pre-trained YOLOv8n model was first evaluated on the test set without any fine-tuning. The model was trained on the COCO dataset, which includes 80 general object classes but does not include specific traffic sign subcategories. Results were extremely poor:

Metric	Score	Interpretation
mAP@50	0.56%	Essentially no correct detections
mAP@50-95	0.25%	Near-zero across IoU thresholds
Precision	1.14%	Almost all detections were wrong
Recall	15.17%	Only found 15% of all signs

The pre-trained model produced almost no detections on the 1,024 test images, confirming that domain-specific fine-tuning is essential for traffic sign detection.

3.2 Fine-tuned YOLOv8n (After Training)

The model was fine-tuned on the Traffic and Road Signs dataset using the following configuration:

- Base model: YOLOv8n (nano — lightweight, suitable for CPU training)
- Epochs: 10
- Image size: 640x640
- Batch size: 8
- Data augmentation: Enabled

After fine-tuning, performance improved dramatically across all metrics:

Metric	Pre-trained	Fine-tuned	Improvement
mAP@50	0.56%	93.3%	+92.74%
mAP@50-95	0.25%	77.9%	+77.65%
Precision	1.14%	91.0%	+89.86%
Recall	15.17%	93.4%	+78.23%

The fine-tuned model achieved over 93% mAP@50, demonstrating that even a short training run (10 epochs) with a relatively small model significantly improves traffic sign detection performance. Sample detection results from the fine-tuned model are shown below:

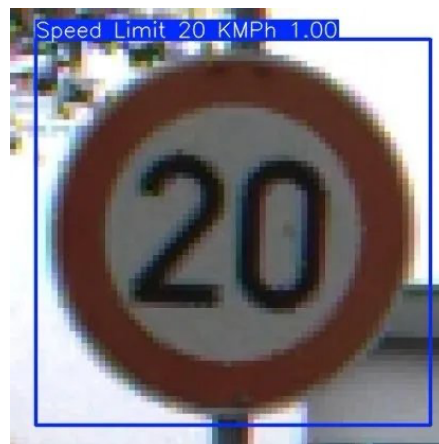


Figure 1: Speed Limit 20 KPh correctly detected with 1.00 confidence

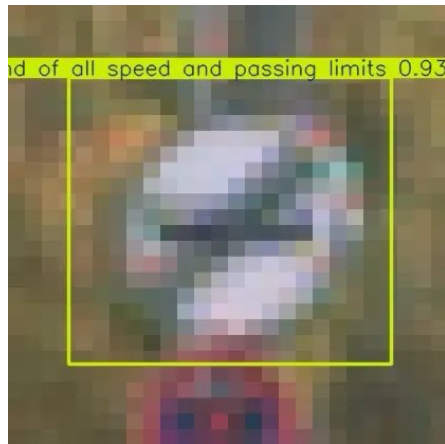


Figure 2: End of all speed and passing limits detected at 0.93 confidence on a blurry image



Figure 3: Misclassification example — 30 KMPH sign labeled as 50 mph speed limit (0.55 confidence), illustrating a limitation of the model on visually similar signs

3.3 Per-Class Performance Highlights

Several classes achieved near-perfect detection scores after fine-tuning:

- Road narrows on right: Precision 0.996, Recall 1.0, mAP@50 0.995
- Attention Please: Precision 1.0, Recall 1.0, mAP@50 0.995
- No Entry: Precision 0.999, Recall 1.0, mAP@50 0.995
- Give Way: Precision 1.0, Recall 0.994, mAP@50 0.995
- Traffic Signal: Precision 0.295, Recall 0.667, mAP@50 0.527 (lowest performer — small object with limited training examples)

3.4 Confidence Threshold and NMS Analysis

To analyze the effect of confidence threshold, inference was run on all 1,024 test images at two different thresholds using the fine-tuned model:

Confidence Threshold	Total Detections	Interpretation
0.3 (low)	1,117	More detections, higher recall, more false positives
0.5 (default)	~950	Balanced precision and recall
0.7 (high)	795	Fewer detections, higher precision, fewer false positives

Lowering the threshold from 0.7 to 0.3 increased detections by approximately 40% (795 to 1,117). For safety-critical applications like autonomous driving, a lower threshold is preferable to avoid missing signs, even at the cost of some false positives. A threshold of 0.5 provides a good balance for general use.

Non-Maximum Suppression (NMS) default settings were used throughout, which effectively prevented duplicate bounding boxes on the same sign. Individual detections such as Speed Limit 20 KMPH were detected with 1.00 confidence, demonstrating the model's reliability on clear, well-lit signs.

4. Suggestions for Improving Small Object Detection

Based on the experimental results, the following strategies could further improve performance:

- Higher image resolution: Training with `imgsz=1280` instead of 640 would provide more pixel detail for small signs. This was tested initially but was too slow for CPU-only training.
- Larger model: Using YOLOv8s or YOLOv8m instead of YOLOv8n would increase model capacity and likely improve detection of harder classes like Traffic Signal.
- More training epochs: 10 epochs gave excellent results, but 30-50 epochs with early stopping could squeeze out additional performance.
- GPU training: Training on a GPU would reduce training time from hours to minutes, allowing for faster experimentation with larger models and resolutions.
- Multi-scale training: Enabling multi-scale augmentation helps the model learn to detect signs at different distances and sizes.
- Anchor box tuning: Custom anchor boxes sized for small traffic signs could improve bounding box regression accuracy.
- Class balancing: The Traffic Signal class had the lowest performance, likely due to fewer training examples. Oversampling underrepresented classes or applying class-weighted loss could help.

5. Conclusion

This assignment demonstrated the effectiveness of fine-tuning a pre-trained YOLO model for domain-specific small object detection. The pre-trained YOLOv8n model achieved essentially zero performance on traffic sign detection ($\text{mAP}@50 = 0.56\%$), while the fine-tuned model achieved 93.3% $\text{mAP}@50$ after just 10 epochs of training on the Traffic and Road Signs dataset. The results confirm that transfer learning with domain-specific data is a highly effective approach for small object detection tasks, even when training is constrained to CPU hardware.